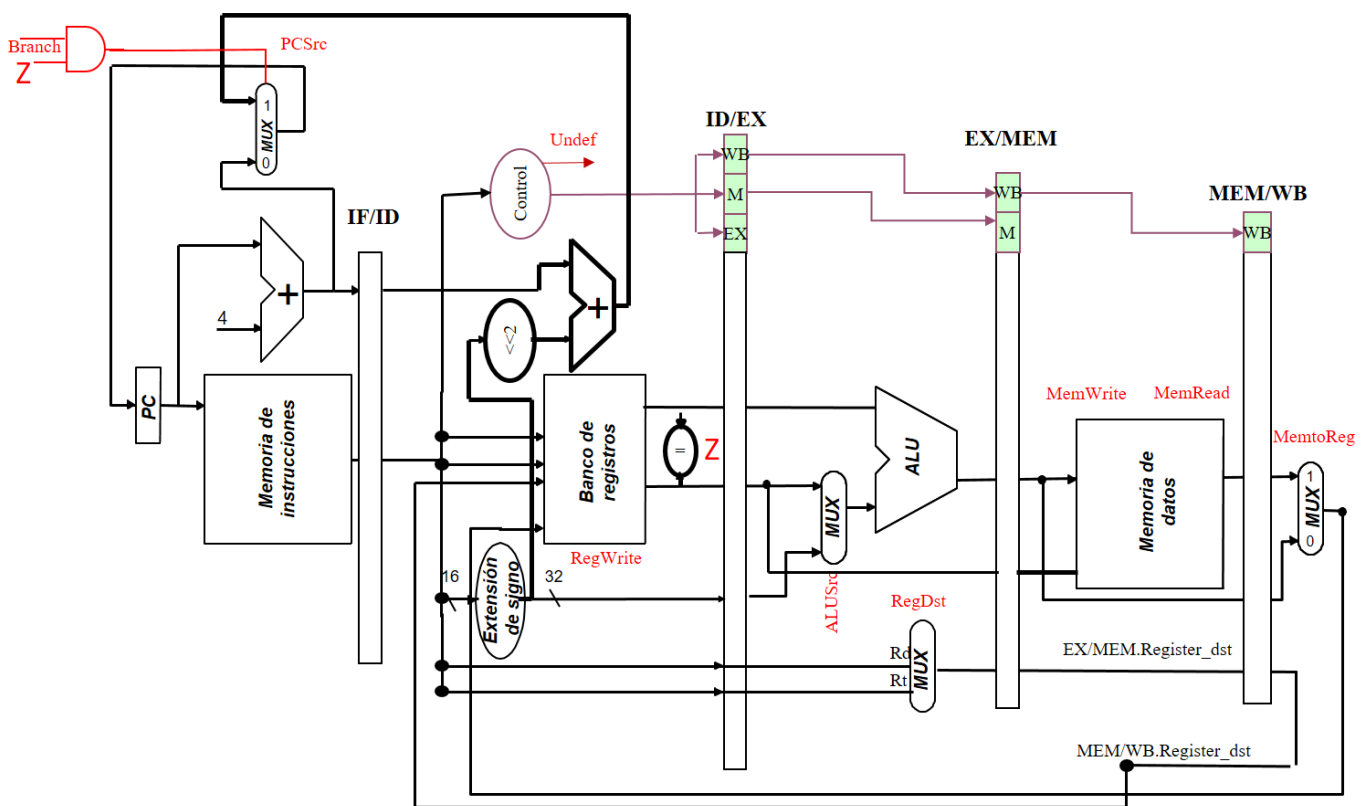


SEGMENTED PROCESSOR FOR A FULLY- DELAYED ISA

ADDING NEW INSTRUCTIONS

Lab Assignment #3



Computer Architecture and Organization 2
Grado en Ing. de Computadores, 2º Curso

Luis M. Ramos Alejandro Valero José Luis Briz Javier Resano
luisma@unizar.es alvabre@unizar.es briz@unizar.es briz@unizar.es jresano@unizar.es



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

1 SUMMARY

In this assignment, you have to modify a hazard-agnostic segmented processor, described in VHDL, in order to execute two new instructions. Then, you have to design a program that tests all the necessary cases, and test the execution using a VHDL simulator.

This assignment reinforces the concepts of segmented processor design studied in theory and problems, and provides experience in design techniques using hardware description languages (VHDL). In addition, it is the first step if you opt-in for trying to pass the course on a continuous evaluation basis, because this is the first approach to the baseline MIPS processor leveraged in the two required course projects.

*First complete the work in sections **2.1.a** and **2.2**. Before starting to work, log in to Moodle from one of the lab computers and **check in the attendance control of the session**.*

*The practice is considered completed when the processor is working properly and you have submitted your modified **vhdl sources plus results of section 3** through **Moodle**.*

2 SEGMENTED PROCESSOR FOR DELAYED ISA

2.1 PRE-LAB 1 – STUDY THE BASELINE SEGMENTED PROCESSOR

- a) Download the VHDL code of the processor from the Moodle resource 'Practical VHDL source code 3'. Study the sources and try to understand how the processor works. Look for the processor components in the code and identify them with the components in the class slides. It is a good idea to make an enlarged copy of the provided data path schematic and copy the principal signal identifiers as declared in VHDL.

Observe the RAM organization: as we are going to use small programs, we will use memories limited to 128 32-bit words. Addresses will still be 32 bits, but we will only decode the 7 least significant bits.

- b) This processor does not handle any hazard. Therefore, before executing the example program you will have to apply static scheduling by reordering the code, **inserting nop instructions when required**. Remember to update the offset of jump instructions, so that they still jump to the same address as before.
- c) Simulate the execution of the example program, using the GHDL workflow you already know from the previous practice. Observe how the signals are transmitted from one stage to another through the stage banks. Check that the registers and memory have the desired value. There is a video in Moodle to help you get started.

2.2 ADDING THE INSTRUCTIONS

```
jal rt, immed16: BR(rt) ← PC + 4; PC ← PC + 4 + 4*SignExt(immed16);  
ret rs: PC ← BR(rs);
```

- a) Draw on the schematic the necessary modifications for the processor to execute the new instructions; specify the corresponding RTL actions.

The purpose of these instructions is to jump to a routine and return to the correct address at the end of its execution. Their implementation requires modifying the logic that updates the PC (mux inputs and control signals). You also have to manage the writing of PC+4 to the register bank: do this in the WB stage, propagating PC+4 up to this stage by making the necessary modifications. You will also have to modify the UC to deal with the new instructions and manage their corresponding control signals.

- b) Draw the MIPS instruction format of your choice for both instructions indicating the use of their fields. Hint: sticking to the MIPS instruction formats is a good option.
- c) Using the previous instructions, write an assembler program (*NIA.txt*) that using a loop loads from memory to register the values obtained in webdiis.unizar.es/~luisma/aoc2/nia2.php from your NIA, and calculates the sum of all of them, finally storing it in the @0. Insert `nop` instructions for the delays.

You must use at least one subroutine, to leverage the `jal` and `ret` instructions. Remember that it is a delayed ISA (the processor ignores hazards) and therefore all delays are visible to the machine language code. In this processor, data RAM and instruction RAM respond in one cycle.

- d) Prepare a second version of your program (*NIA-R.txt*) by reordering the code using static planning, in order to minimize execution time.

2.3 DESIGN IN VHDL

- a) Write in VHDL the modifications you have devised to implement `jal` and `ret`. Add a comment with your name and Unizar *email* in the header of all the vhd files you modify.
- b) Encode the instructions of your programs *NIA.txt* and *NIA-R.txt* in instruction RAM files *RAM_I_NIA.vhd* and *RAM_I_NIA-R.vhd*. Initialize the data RAM (*RAM_NIA.vhd*) taking into account the above.

Use only the following operation codes and no others:

<code>jal: 000101</code>	<code>ret: 000110</code>
--------------------------	--------------------------

Encoding of ALU operations (funct field and ALU_ctrl signal):

<code>+: 00000</code>	<code>-: 00001</code>	<code>AND: 00010</code>	<code>OR: 00011</code>
-----------------------	-----------------------	-------------------------	------------------------

- c) Mind the problem of the return address when returning with `ret` to an address saved by `jal`. Briefly explain your solution in the header of *NIA.txt*
- d) Simulate the execution, and debug until the program works correctly.
- e) Submit to Moodle a *NIA.zip* file with the complete source files and the *NIA.txt* and *NIA-R.txt* files.

3 PERFORMANCE ANALYSIS

- a) Calculate the *speedup* of your optimized code using first the time and then the speed in MIPS. Explain the differences.
- b) In view of the tex decomposition expression ($CPI \cdot N_i \cdot t_c$), what is the key factor in the improvement in this case?

